National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	04
Course Code:	CL2001
Course Title:	Data Structures - Lab
Semester:	Fall 2026
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk

Guidelines

- You may use **only the concepts covered in the lab** to complete this task.
- Your submission must be a **single PDF file** that includes:
  - The source code (in plain text)
  - Screenshots of the corresponding outputs
- Name your file using the following format: **rollno_name_labtasknumber.pdf**
  Example: *25p-1234_ozair_labtask04.pdf*
- Unethical use of AI tools will result in **0 marks**.
- Late submissions will incur a penalty of **-3 marks per day**.

Tasks

ONLY USE POINTER NOTATION TO IMPLEMENT ALL THE SOLUTIONS. ARRAY INDEXING [ ] IS NOT ALLOWED.

Section 1: Class Member Functions

1 Convert the Linked List to Store Character Values

Modify the existing `LinkedList` implementation so that it stores `char` values instead of integers. Update the node data type, data members, constructors, and all relevant member functions accordingly.

2 Add and Maintain a Tail Pointer

Modify the `LinkedList` implementation to maintain both a `head` pointer and a `tail` pointer. The `tail` pointer should always point to the last node in the list. Update the data members, constructors, and all relevant member functions to correctly maintain both pointers after insertions, deletions, and other list operations.

3 Implement replaceHead()

Implement a member function named `replaceHead()` that moves the last node of the linked list to the front of the list.

The operation must update the node links appropriately rather than creating a new node or simply swapping data values.

Example:

```
Given: a->b->c->d->End
Output: d->a->b->c->End
```

4 Implement removeDuplicates()

Implement a member function named `removeDuplicates()` that removes duplicate values from the linked list. Assume that the list is already sorted in ascending order. The function must remove duplicate nodes in **linear time**.
Make sure that the `head` and `tail` pointers are updated correctly when duplicate nodes are removed.

Example:

```
Given: a->a->b->c->d->d->d->e->End
Output: a->b->c->d->e->End
```

5 Implement clear()

Implement a member function named `clear()` that removes all nodes from the linked list and leaves the list empty. After the operation, both `head` and `tail` should indicate that the list contains no elements.

6 Implement reverse()

Implement a member function named `reverse()` that reverses the linked list **in place**.
The function must modify the existing node links/pointers to reverse the list. Do **not** create a separate linked list or swap/replace the data values stored in the nodes.
After reversal, ensure that both the `head` and `tail` pointers correctly point to the new first and last nodes, respectively.

Section 2: Non-Member Functions

1 ArrayList to LinkedList

Write a non-member function that takes an `ArrayList` object as its only parameter, converts its elements into a newly created singly linked list, and returns the resulting `LinkedList` object.

The function must use **only the public member functions** provided by the `ArrayList` and `LinkedList` classes. Direct access to their private data members is not allowed.

Example:

```
Given: [1,3,5,6]
Output: 1->3->5->6->End
```

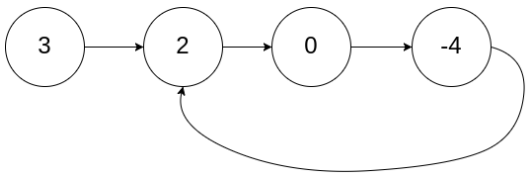
2 Linked List Cycle

Write a non-member function that takes the `head` pointer of a linked list as its parameter and determines whether the linked list contains a cycle.
A cycle exists if, by continuously following the `next` pointers, a previously visited node can be reached again.
The function should:

- Return `true` if the linked list contains a cycle.
- Return `false` if the linked list does not contain a cycle.

You may access the private data members of the `LinkedList` class by declaring the function as a **friend function** of the class.

Examples:




```
Input: list = 3->2->0->-4->2->...
Output: true

Input: head = 1->2->1->...
Output: true

Input: list1 = 1->6->1->End
Output: false
```

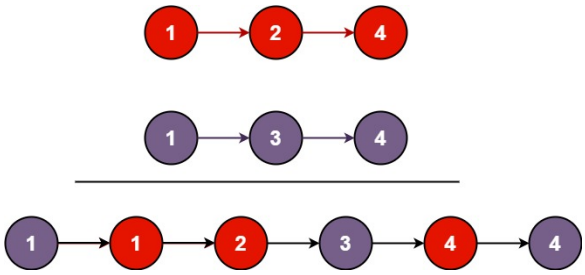
3 Merge Two Sorted Lists

Write a non-member function that takes two `LinkedList` objects as parameters. Both linked lists are sorted in ascending order.
The function should merge the two lists into a single sorted linked list by **splicing together the existing nodes** from the two input lists.
The function must satisfy the following requirements:

- The merged list must maintain ascending order.
- Do **not** create or use a third temporary linked list.
- Modify the existing node links (`next` pointers) to perform the merge.
- Return the `head` pointer of the resulting merged linked list.

You may access the private data members of the `LinkedList` class by declaring the function as a **friend function** of the class.

Examples:




```
Input: list1 = 1->2->4->End, list2 = 1->3->4->End
Output: 1->1->2->3->4->4->End

Input: list1 = Empty, list2 = Empty
Output: Empty

Input: list1 = Empty, list2 = 0->End
Output: 0->End
```